K A S U

Security Audit Report

RWA Private Credit Lending Platform

Auditor: Claude by Anthropic (AI-Assisted Review)

Date: April 9, 2026

Commit: `b18e98716c3720e821699f081520484648d6d729`

Branch: release-candidate

Solidity Version: 0.8.23

Scope: 40 contracts + 2 vendor · ~9,728 LoC · Full + Lite deployments

AI-Assisted Security Review — Not a Replacement for Professional Audit

kasu.finance · Docs · Twitter/X · Discord · Telegram · Medium

IMPORTANT NOTICE

Disclaimer

Disclaimer

This security audit was performed by Claude (Anthropic), an AI assistant. While AI-assisted reviews can identify many classes of vulnerabilities and provide valuable security insights, this report has inherent limitations:

- AI models may miss subtle vulnerabilities that require deep domain expertise or novel attack vectors.
- This review is based on static analysis of source code. No dynamic testing, formal verification, or fuzzing was performed.
- Cross-contract interactions and deployment-specific configurations may not be fully captured.
- This report should be treated as a supplement to, not a replacement for, a professional security audit by an established firm.
  The Kasu team is strongly encouraged to engage a professional audit firm (e.g., Trail of Bits, OpenZeppelin, Cyfrin, Spearbit) for a comprehensive review before mainnet launch or significant upgrades.

All findings reflect the auditor's assessment at the time of review and are subject to the constraints above.

NAVIGATION

Table of Contents

Executive Summary

Scope & Methodology

Assumptions & Trust Model

Findings Summary

High Severity Findings

Medium Severity Findings

Low Severity Findings

Informational Findings

Security Checklist

Go / No-Go Recommendation

Appendix A: Contract Inventory

Appendix B: Comparison with February 2026 Audit

SECTION 1

Executive Summary

Overall Risk Assessment:

MEDIUM

The Kasu protocol demonstrates strong security engineering with comprehensive role-based access control, proper use of OpenZeppelin v5 libraries, epoch-based design that eliminates most MEV vectors, and defense-in-depth patterns throughout. This second audit, conducted using three complementary methodologies (Feynman deep logic analysis, State Inconsistency structural analysis, and Nemesis iterative fusion), confirms the protocol's solid architectural foundation while surfacing one high-severity state inconsistency bug and several medium-severity issues warranting attention.

0	1	8	6	7
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

Top Risks

UserManager stale `_isUserPartOfLendingPool` mapping corrupts re-enrollment — After pool removal, the mapping is never reset to `false`, causing re-depositing users to be invisible to loyalty calculations and potentially gaining unfair clearing priority (H-01).

Fee emission permanently lost when no eligible rKSU holders — USDC fees transferred to KSULocking when `eligibleRKSUForFees == 0` are permanently stranded with no recovery mechanism (M-01).

Admin privilege risks without guardrails — `recoverERC20` can drain KSU reward pools, `emitUserLoyaltyRewardBatch` allows arbitrary reward inflation, config changes take immediate effect without timelock (M-02, M-03, M-08).

Loss repayment has no upper bound — `recoveredAmount` can exceed `lossAmount`, enabling over-recovery that distributes excess funds to loss token holders (M-05).

Balance-based swap accounting — `DepositSwap._transferAndSwap` uses absolute `balanceOf` instead of delta, potentially crediting residual tokens to swap depositors (M-07).

SECTION 2

Scope & Methodology

Scope

All Solidity implementation contracts in the `src/` directory and vendored dependencies in `vendor/`, covering all deployment modes (Base Full, Plume Lite, XDC Lite).

Category	Files	Lines of Code
Core — Lending Pool	10 files	~4,560
Core — Clearing	5 files	~1,533
Core — Config & Management	8 files	~1,850
Core — Swap	2 files	~279
Locking System	4 files	~656
Token	2 files	~212
Access Control	3 files	~265
Lite Variants	5 files	~246
Shared Utilities	3 files	~127
Vendor (Nexera)	2 files	~293
Total	**42 + 2 vendor**	**~9,728 + 293**

Methodology

This audit employed three complementary methodologies executed in parallel, with a final iterative fusion pass:

1. Feynman Deep Logic Audit

First-principles reasoning applied to every function. Questions every line's purpose, ordering, guards, and implicit assumptions. Finds business logic bugs that pattern-matching and checklists miss. Applied across 3 parallel workstreams covering all 42 contracts.

2. State Inconsistency Audit

Structural invariant analysis mapping all coupled state pairs, building mutation matrices, and cross-checking every mutation path for missing state synchronization. 8-phase process covering dependency mapping, parallel path comparison, user journey tracing, and masking code detection. 15 coupled state pairs mapped, ~50 mutation paths analyzed.

3. Nemesis Fusion Pass

Iterative cross-pollination of Feynman and State Inconsistency findings. Tests intersection hypotheses where bugs from different methodologies compound. 3 intersection hypotheses investigated; 2 confirmed as true intersections (non-amplifying), 1 eliminated.

SECTION 3

Assumptions & Trust Model

Privileged Roles

Role	Holder	Capabilities
`DEFAULT_ADMIN_ROLE` / `ROLE_KASU_ADMIN`	Kasu Multisig	Pause system, configure fees/thresholds, manage roles, recover tokens, block users
`ROLE_POOL_ADMIN`	Pool Admin Multisig	Create pools, configure pool parameters, manage pool roles
`ROLE_POOL_MANAGER`	Pool Manager Multisig	Update draw amounts, repay funds, report losses
`ROLE_POOL_CLEARING_MANAGER`	Pool Admin Multisig	Execute clearing process
`ROLE_PROTOCOL_FEE_CLAIMER`	Configured address	Claim accumulated protocol fees

Upgradeability

Transparent Proxy (per-proxy ProxyAdmin, OZ v5) for core singletons. Beacon Proxy for pool instances (LendingPool, PendingPool, LendingPoolTranche). Each proxy has its own dedicated ProxyAdmin contract. All ProxyAdmin and Beacon ownership is (or should be) held by the Kasu multisig.

External Dependencies

USDC — Underlying asset for all lending pools. Trusted as non-reentrant ERC20.
Compilot (NexeraID) — KYC/KYB signature gating. Off-chain signer controlled by third party.
OpenZeppelin Contracts v5.0.2 — Base contracts for access control, tokens, proxies.

Trust Model

Admin multisigs are trusted. Inter-contract calls between Kasu system contracts (LendingPoolManager, ClearingCoordinator, PendingPool, LendingPool, LendingPoolTranche, FixedTermDeposit) are trusted. End users interact only through LendingPoolManager. KYC signatures are validated on-chain via the vendored NexeraID verifier.

SECTION 4

Findings Summary

ID	Severity	Title	Contract
H-01	HIGH	Stale `_isUserPartOfLendingPool` mapping corrupts user re-enrollment and clearing priority	UserManager
M-01	MEDIUM	Fee emission permanently lost when no eligible rKSU holders	KSULocking
M-02	MEDIUM	`recoverERC20` can drain KSU reward pool beyond unclaimed balance	UserLoyaltyRewards
M-03	MEDIUM	`emitUserLoyaltyRewardBatch` allows arbitrary reward inflation	UserLoyaltyRewards
M-04	MEDIUM	Stale KSU epoch price inflates loyalty levels when `updateKsuEpochTokenPrice` not called	SystemVariables
M-05	MEDIUM	Loss repayment has no upper bound; `recoveredAmount` can exceed `lossAmount`	LendingPoolTrancheLoss
M-06	MEDIUM	Incomplete loss token minting blocks clearing step 4	LendingPoolTrancheLoss
M-07	MEDIUM	Balance-based swap accounting includes residual tokens	DepositSwap
M-08	MEDIUM	Withdrawal pro-rata truncation to zero permanently delays small positions	AcceptedRequestsExecution
L-01	LOW	Max deposit check only applies to pending deposits, not total position	PendingPool
L-02	LOW	Deposit round-up can aggregate dust-level over-acceptance across priority buckets	AcceptedRequestsExecution
L-03	LOW	`batchSize=0` accepted silently as no-op in clearing steps	PendingRequestsPriorityCalc
L-04	LOW	Swapper `safeIncreaseAllowance` overflow risk with non-standard tokens	Swapper
L-05	LOW	Duplicate loyalty threshold values allowed in configuration	SystemVariables

ID	Severity	Title	Contract
L-06	LOW	`_totalUserTranchePendingDepositAmount` not tracked against total position	PendingPool
I-01–I-07	INFO	7 informational findings (see section 8)	Various

SECTION 5

High Severity Findings

H-01 HIGH Stale `_isUserPartOfLendingPool` mapping corrupts user re-enrollment and clearing priority

CONTRACT `UserManager.sol` — `_removeLendingPoolFromUser()` (lines 497–499)
METHODOLOGY State Inconsistency Audit (SI-001) + Feynman Audit (F-02) — cross-confirmed

IMPACT

When a lending pool is removed from a user's `_userLendingPools` array via swap-and-pop in `_removeLendingPoolFromUser()`, the `_isUserPartOfLendingPool[lendingPool][user]` mapping is **never reset to `false`**. There is no code path in the entire codebase that sets this mapping to `false`. If the user later deposits into the same pool, the guard at line 314 incorrectly skips re-adding the pool to `_userLendingPools[user]`, leaving the array empty.

EXPLOIT SCENARIO

1. Alice deposits into Pool X. `_isUserPartOfLendingPool[X][Alice] = true`, `_userLendingPools[Alice] = [X]`.
2. Alice fully withdraws. Admin calls `updateUserLendingPools()`. Pool X is removed from Alice's array via swap-and-pop, but `_isUserPartOfLendingPool[X][Alice]` remains `true`.
3. Alice deposits into Pool X again. `userRequestedDeposit()` checks `_isUserPartOfLendingPool[X][Alice]` — it's `true`, so the push is **skipped**. `_userLendingPools[Alice]` remains empty.
4. `_userTotalPendingAndActiveDepositedAmount(Alice)` iterates the empty array and returns 0.
5. **If Alice has rKSU:** `rKSUDepositRatio = type(uint256).max` (division by zero guard), giving Alice maximum loyalty level and unfair clearing priority, potentially displacing other users' deposits/withdrawals.
6. **If Alice has no rKSU:** her deposit amount is reported as 0 to the loyalty reward system. She receives zero loyalty rewards despite having an active deposit — permanent loss of earned rewards.

ROOT CAUSE

Missing `_isUserPartOfLendingPool[pool][user] = false` in the removal function. The only write to this mapping is `= true` at line 316.

RECOMMENDED FIX

```
function _removeLendingPoolFromUser(address user, uint256 userLendingPoolIndex)
    address lendingPool = _userLendingPools[user][userLendingPoolIndex];
    _isUserPartOfLendingPool[lendingPool][user] = false;  // ADD THIS LINE
    _userLendingPools[user][userLendingPoolIndex] =
        _userLendingPools[user][_userLendingPools[user].length - 1];
    _userLendingPools[user].pop();
}
```

TEST TO ADD

User deposits, fully withdraws, `updateUserLendingPools` is called, user re-deposits into the same pool. Verify pool is tracked in `_userLendingPools`, loyalty level is correct, and clearing priority is fair.

SECTION 6

Medium Severity Findings

M-01 MEDIUM Fee emission permanently lost when no eligible rKSU holders

CONTRACT `KSULocking.sol` — `emitFees()` / `_updatePoolRewards()` (line 427)

IMPACT

When `eligibleRKSUForFees == 0`, USDC fees are transferred to KSULocking via `safeTransferFrom` but `_updatePoolRewards()` silently returns without updating `_accumulatedRewardsPerShare`. These tokens are permanently stuck with no recovery mechanism. When users later enable fees, they will never receive these tokens because the per-share accumulator was never incremented.

ROOT CAUSE

`_updatePoolRewards` silently returns when no eligible recipients exist, but tokens were already transferred.

RECOMMENDED FIX

Revert in `emitFees` when `eligibleRKSUForFees == 0`, or buffer unallocated fees for later distribution, or redirect to protocol fee receiver.

TEST TO ADD

Test `emitFees` when no users are fee-eligible. Verify funds are not permanently lost.

M-02 MEDIUM `recoverERC20` can drain KSU reward pool beyond unclaimed balance

CONTRACT `UserLoyaltyRewards.sol` — `recoverERC20()` (lines 196–198)

IMPACT

The admin can transfer ANY ERC20 from the contract, including KSU tokens backing unclaimed user rewards. No guard prevents withdrawing below `totalUnclaimedRewards`. Users would be unable to claim their accrued KSU rewards as the `ksuBalance` check at line 170–173 caps claims at the actual balance.

ROOT CAUSE

Missing check: `tokenAmount ≤ balance - totalUnclaimedRewards` for the KSU token.

RECOMMENDED FIX

Add: `if (tokenAddress == address(_ksuToken)) require(tokenAmount ≤ _ksuToken.balanceOf(address(this)) - totalUnclaimedRewards)`.

TEST TO ADD

Verify `recoverERC20` reverts when attempting to withdraw KSU below the unclaimed rewards threshold.

M-03 MEDIUM `emitUserLoyaltyRewardBatch` allows arbitrary reward inflation

CONTRACT `UserLoyaltyRewards.sol` — `emitUserLoyaltyRewardBatch()` (lines 140–157)

IMPACT

Admin can call with arbitrary parameters (any user, any `amountDeposited`, any `ksuTokenPrice`) without on-chain validation against actual state. A compromised admin multisig could inflate rewards and drain all KSU from the contract.

ROOT CAUSE

Batch emission function trusts admin input without on-chain state validation.

RECOMMENDED FIX

Add a maximum reward cap per call. Ensure admin is a well-configured multisig with timelock. Document this as an intentional admin privilege.

TEST TO ADD

Test that extreme parameter values produce bounded reward amounts.

M-04 MEDIUM Stale KSU epoch price inflates loyalty levels when `updateKsuEpochTokenPrice` not called

CONTRACT `SystemVariables.sol` (lines 235–246) / `UserManager.sol` (line 444)

IMPACT

`updateKsuEpochTokenPrice()` must be called each epoch by an external caller. If not called, `ksuEpochTokenPrice` remains at its last-updated value. The loyalty level calculation in `UserManager._userLoyaltyLevel()` uses this stale price. If KSU price has dropped since the last update, users' rKSU appears worth more than it is, inflating loyalty levels and giving unfair clearing priority and higher KSU rewards.

ROOT CAUSE

No enforcement that `updateKsuEpochTokenPrice()` is called every epoch. No automated mechanism or clearing-time integration.

RECOMMENDED FIX

Call `updateKsuEpochTokenPrice()` at the start of `batchCalculateUserLoyaltyLevels`, or use the live price from `_ksuPrice.ksuTokenPrice()` directly instead of the epoch-cached value.

TEST TO ADD

Skip `updateKsuEpochTokenPrice` for several epochs, verify loyalty levels use stale price. Fix and verify live price is used.

M-05 MEDIUM Loss repayment has no upper bound; `recoveredAmount` can exceed `lossAmount`

CONTRACT `LendingPoolTrancheLoss.sol` — `repayLoss()` (line 140)

IMPACT

The `repayLoss` function increments `recoveredAmount` without checking that it doesn't exceed the original `lossAmount`. If `recoveredAmount > lossAmount`, the `userClaimableLoss()` calculation distributes more than the actual loss to token holders, draining excess USDC from the tranche contract.

ROOT CAUSE

No upper bound validation: `_lossDetails[lossId].recoveredAmount += amount` with no cap.

RECOMMENDED FIX

```
if (_lossDetails[lossId].recoveredAmount + amount > _lossDetails[lossId].lossAm
    revert RecoveryExceedsLoss(lossId);
}
```

TEST TO ADD

Call `repayLoss` with cumulative amounts exceeding original loss. Verify it reverts.

M-06 MEDIUM Incomplete loss token minting blocks clearing step 4

CONTRACT `LendingPoolTrancheLoss.sol` (modifier `notPendingLossMint`) + `LendingPoolTranche.sol` (lines 139, 170)

IMPACT

When a loss is reported with `doMintLossTokens = false`, the `notPendingLossMint` modifier blocks all `deposit()` and `redeem()` on the tranche until batch minting completes. Clearing step 4 calls these functions via `acceptDeposit` / `acceptWithdrawal`, causing clearing to revert with `LossMintingInProgress`. For tranches with many users, minting may take multiple transactions.

ROOT CAUSE

`ClearingCoordinator.doClearing` does not check for pending loss mints before attempting step 4. The `reportLoss` function requires `verifyClearingNotPending`, but there is a window between loss report and next clearing where minting must complete.

RECOMMENDED FIX

Consider having the clearing coordinator check for pending loss mints and either complete them or skip the affected tranche. Alternatively, require `doMintLossTokens = true` for production usage and document operational guidelines.

TEST TO ADD

Report loss with `doMintLossTokens = false`, attempt clearing. Verify clearing reverts. Complete minting and verify clearing proceeds.

M-07 MEDIUM Balance-based swap accounting includes residual tokens

CONTRACT `DepositSwap.sol` — `_transferAndSwap()` (line 96)

IMPACT

The function measures swap output as `IERC20(outToken).balanceOf(address(this))` after the swap. If LendingPoolManager holds residual USDC from a prior transfer or donation, that amount is included in the deposit, allowing a user to deposit more than their swap actually produced.

ROOT CAUSE

Absolute balance check instead of delta-based accounting (balance before vs after).

RECOMMENDED FIX

Record `balanceBefore` before the swap. Compute `outTokenAmount = balanceOf(this) - balanceBefore`.

TEST TO ADD

Send USDC directly to LendingPoolManager, then execute a swap deposit. Verify only the swapped amount is deposited.

M-08 MEDIUM Withdrawal pro-rata truncation to zero permanently delays small positions

CONTRACT `AcceptedRequestsExecution.sol` — line 178

IMPACT

When computing accepted withdrawal shares pro-rata (`userWithdrawalShares * totalAcceptedWithdrawalShares / totalRequestedWithdrawalShares`), small positions can truncate to 0. The withdrawal NFT survives with full shares amount, and the user gets 0 shares accepted. This repeats every epoch since the ratio doesn't change for that user's position — the withdrawal request is effectively stuck permanently.

ROOT CAUSE

Integer division truncation with no minimum acceptance threshold or dust-clearing mechanism.

RECOMMENDED FIX

Either set a minimum accepted amount (e.g., accept fully if shares < dust threshold), or round up for the smallest positions, or allow users to cancel and re-submit.

TEST TO ADD

Create a withdrawal request with a very small share amount relative to total requested. Verify it eventually gets processed (not stuck at 0 forever).

SECTION 7

Low Severity Findings

ID	Title	Contract
L-01	Max deposit check only applies to pending deposits, not total user position. Repeated epoch deposits can exceed `maxDepositAmount`.	PendingPool
L-02	Deposit round-up (`Math.ceilDiv`) can aggregate dust-level over-acceptance across priority buckets. Accepted deposit may exceed `totalAcceptedDepositAmount` by small amounts.	AcceptedRequestsExecution
L-03	`batchSize=0` accepted silently as no-op in clearing steps 2 and 4. No progress is made, wasting gas without warning.	PendingRequestsPriorityCalc / AcceptedRequestsExec
L-04	`Swapper._approveMax` uses `safeIncreaseAllowance(type(uint256).max)`. If `_resetApproval` fails to clear prior allowance (non-standard token), the next call overflows and reverts.	Swapper
L-05	`_setLoyaltyThresholds` uses strict `>` (not `≥`) comparison, allowing duplicate adjacent threshold values. Equal thresholds collapse loyalty levels in potentially unintended ways.	SystemVariables
L-06	`_totalUserTranchePendingDepositAmount` tracks pending deposits per user but does not account for existing tranche shares when checking `maxDepositAmount`. Users can exceed intended limits by depositing across epochs.	PendingPool

SECTION 8

Informational Findings

ID	Title	Contract
I-01	Zero-amount protocol fee claim succeeds with misleading `ProtocolFeesClaimed` event (no-op transfer)	FeeManager
I-02	Asymmetric grant/revoke access: Factory can grant but not revoke pool roles; Kasu Admin can revoke but not grant (by design)	KasuController
I-03	`emitUserLoyaltyReward` access check at line 95 occurs after early return at line 91, allowing unauthorized callers when `doEmitRewards` is false (no state change)	UserLoyaltyRewards

ID	Title	Contract
I-04	Integer division rounding on partial KSU unlocks causes rKSU dust accumulation. Over many partial withdrawals, fractions of rKSU may be lost.	KSULocking
I-05	ERC4626 inflation attack mitigated by 12-decimal offset, making the attack economically infeasible	LendingPoolTranche
I-06	`feesOwedAmount -= overpaidFees` could theoretically underflow in extreme edge case with single-depositor pool and near-zero fixed rate	LendingPool
I-07	Non-transferable tokens (PendingPool NFTs, tranche shares, loss tokens) correctly enforced via function overrides — strong security property	Various

SECTION 9

Security Checklist

PASS	**Access Control**	Comprehensive role-based access via KasuController. All state-changing functions guarded by modifiers (`onlyLendingPoolManager` , `onlyClearingCoordinator` , `onlyPendingPool` , etc.). Pool-scoped roles via KasuAccessControllable. One stale mapping issue (H-01).
PASS	**Reentrancy**	No explicit reentrancy guards, but underlying asset is USDC (no transfer hooks). All sensitive functions access-controlled to trusted system contracts. SafeERC20 used throughout. CEI pattern generally followed.
PASS	**Arithmetic**	Solidity 0.8.23 provides checked arithmetic. `unchecked` blocks used judiciously for loop counters. Interest rates use `INTEREST_RATE_FULL_PERCENT = 1e18` . ERC4626 offset of 12 decimals prevents inflation. Minor rounding dust is acceptable.
PASS	**External Calls**	All token transfers via SafeERC20. No low-level `call` to untrusted addresses (Swapper uses `Address.functionCall` to allowlisted exchanges only). Return values properly checked.
PASS	**Upgradability**	TransparentProxy with per-proxy ProxyAdmin (OZ v5). BeaconProxy for pool instances. `_disableInitializers()` called in constructors. `initializer` modifier on all init functions. Storage gaps in KasuAccessControllable.
PASS w/ NOTE	**Oracle Dependencies**	No DeFi price oracles for core lending. KSU price is admin-set or fetched from KsuPrice contract (only affects loyalty, not accounting). `updateKsuEpochTokenPrice` is permissionless but once-per-epoch and can go stale (M-04).
PASS	**MEV Exposure**	Epoch-based clearing eliminates real-time MEV. Deposits/withdrawals queued and processed at epoch boundaries. Clearing executed by permissioned roles. Swap deposits have `minAmountOut` slippage protection.
PASS	**ERC Compliance**	ERC20 (LendingPool, KSU), ERC721 (PendingPool NFTs), ERC4626 (LendingPoolTranche), ERC1155 (loss tokens, ExternalTVL) follow OpenZeppelin implementations. Non-standard behavior (non-transferable tokens, restricted operations) is intentional.
PASS	**State Consistency**	15 coupled state pairs mapped; 1 confirmed desynchronization (H-01). Clearing pipeline maintains consistent state transitions through multi-step process. Reverse-iteration swap-and-pop patterns verified correct.

SECTION 10

Go / No-Go Recommendation

Recommendation: CONDITIONAL GO

The Kasu protocol has a well-engineered architecture with strong defense-in-depth patterns. The permissioned, epoch-based design eliminates most common DeFi attack vectors. No critical vulnerabilities were found. However, the following conditions should be met before production upgrades:

Must Fix (before upgrade)

ID	Finding	Rationale
H-01	UserManager stale pool mapping	Directly corrupts clearing priority and loyalty calculations. One-line fix with high impact. Affects all users who withdraw and re-deposit into the same pool.
M-07	DepositSwap balance-based accounting	Can cause incorrect deposit amounts for swap users. Simple delta-based fix.
M-05	Loss repayment no upper bound	Can cause excess USDC distribution. Simple bounds check.

Should Fix (before next audit cycle)

ID	Finding	Rationale
M-01	Fee emission lost permanently	USDC can be permanently stuck in KSULocking.
M-02	recoverERC20 reward pool drainage	Admin privilege risk mitigated by multisig but should be bounded.
M-04	Stale KSU epoch price	Operational risk; consider automating the price update.
M-06	Loss minting blocks clearing	Operational DoS risk; document guidelines for `doMintLossTokens` parameter.
M-08	Withdrawal pro-rata truncation	Affects small positions; consider dust-clearing mechanism.

Positive Observations

ERC4626 inflation well-mitigated — 12-decimal offset makes inflation attacks economically infeasible.
Comprehensive verification in clearing — `_verifyResult` in AcceptedRequestsCalculation provides thorough post-computation invariant checking.
Well-designed batch processing — Steps 1, 2, and 4 support batching to prevent gas-limit DoS.
Proper use of OpenZeppelin v5 — Per-proxy ProxyAdmin, SafeERC20, AccessControl, ERC4626 with virtual assets.
Epoch-based design eliminates MEV — No real-time pricing, no flash loan vectors, no sandwich attacks.
Clean Lite variant architecture — Lite contracts properly stub out token-dependent functionality while preserving KYC and core deposit flows.
Strong coupled state consistency — Of 15 mapped coupled state pairs, only 1 has a synchronization gap. Clearing pipeline, fee distribution, and locking reward math are structurally sound.
Non-transferable tokens — PendingPool NFTs, tranche shares, and loss tokens are all correctly restricted, eliminating secondary market manipulation vectors.

SECTION 11

Appendix A: Contract Inventory

Core — Lending Pool

Contract	LoC	Description
LendingPool.sol	1,229	Pool ledger, ERC20 token, interest/loss/draw accounting
LendingPoolManager.sol	836	Entry point for all user and admin interactions
PendingPool.sol	823	ERC721 pending request management, extends ClearingSteps
FixedTermDeposit.sol	817	Fixed-rate locked deposits with clearing integration
LendingPoolTrancheLoss.sol	298	ERC1155 impairment receipts, loss registration and repayment
LendingPoolTranche.sol	270	ERC4626 vault for tranche shares
LendingPoolFactory.sol	182	BeaconProxy-based pool deployment
LendingPoolStoppable.sol	~40	Pool stopping mechanism
LendingPoolHelpers.sol	62	Shared helper functions
UserRequestIds.sol	~50	Compact ID encoding library

Core — Clearing

Contract	LoC	Description
AcceptedRequestsCalculation.sol	459	Pure calculation of accepted deposits/withdrawals
ClearingCoordinator.sol	334	Orchestrates 5-step clearing pipeline
PendingRequestsPriorityCalculation.sol	287	Priority calculation by loyalty level
AcceptedRequestsExecution.sol	255	Pro-rata execution of accepted requests
ClearingSteps.sol	197	Abstract contract bridging clearing to PendingPool

Core — Configuration & Management

Contract	LoC	Description
UserManager.sol	543	Loyalty level calculation, user tracking
SystemVariables.sol	470	Global system config (epochs, fees, KSU price)
UserLoyaltyRewards.sol	242	KSU rewards based on liquidity and loyalty
KasuPoolExternalTVL.sol	162	ERC1155 for off-chain TVL tracking
DepositSwap.sol	140	Swap-and-deposit functionality
Swapper.sol	139	DEX swap execution with allowlist
KasuAllowList.sol	132	KYC verification via NexeraID signatures
FeeManager.sol	101	Ecosystem/protocol fee splitting

Locking System

Contract	LoC	Description
KSULocking.sol	474	KSU token locking with fee distribution
KSULockingLite.sol	102	Stub locking for token-free deployments
rKSU.sol	43	Non-transferable receipt token
KSULockBonus.sol	37	Bonus KSU distribution for lockers

Token & Access Control

Contract	LoC	Description
KSU.sol	50	ERC20 token with permit and burn
KasuController.sol	139	Role management, system pause
KasuAccessControllable.sol	101	Base contract for pool-scoped access
Roles.sol	25	Role constant definitions

Lite Variants

Contract	LoC	Description
UserManagerLite.sol	60	Disabled loyalty calculations
ProtocolFeeManagerLite.sol	37	Protocol-only fee handling

Contract	LoC	Description
UserLoyaltyRewardsLite.sol	33	Disabled reward emissions
KsuPriceLite.sol	14	Returns 0 for KSU price

Vendor

Contract	LoC	Description
vendor/nexera/BaseTxAuthDataVerifier.sol	215	Core KYC signature verification
vendor/nexera/TxAuthDataVerifierUpgradeable.sol	78	Upgradeable wrapper for KYC verifier

SECTION 12

Appendix B: Comparison with February 2026 Audit

This audit covers the same codebase as the February 2026 audit (no Solidity changes between commits). The table below compares findings:

Feb 2026 Finding	Apr 2026 Status	Notes
H-01: FTD batch array corruption	Not reproduced	Reverse iteration swap-and-pop verified correct by State Inconsistency audit (FP-2). Element swapped from end was already processed in backward pass.
H-02: Stale nextIndexToProcess	Not reproduced	Related to H-01; same analysis applies. Index is re-initialized at clearing start.
M-01: UserManager stale mapping	**Confirmed as H-01**	Elevated to HIGH. State Inconsistency audit provided deeper analysis showing clearing priority manipulation via `type(uint256).max` loyalty ratio.
M-02: recoverERC20 drain	**Confirmed as M-02**	Independently rediscovered.
M-03: Batch reward inflation	**Confirmed as M-03**	Independently rediscovered.
M-04: Unbounded _allUsers	Acknowledged	Informational — operational concern, not a direct vulnerability.
M-05: block.number KYC expiry	Acknowledged	Vendored dependency (NexeraID). Out of Kasu's direct control.
M-06: Balance-based swap	**Confirmed as M-07**	Independently rediscovered.
M-07: Loss minting blocks ops	**Confirmed as M-06**	Independently rediscovered with additional clearing interaction analysis.
M-08: FTD fallback wrong deposit	Acknowledged	Related to H-01/H-02 analysis; less severe if H-01 swap-and-pop is correct.
M-09: Fee loss no holders	**Confirmed as M-01**	Independently rediscovered.
M-10: ERC1155Burnable bypass	Acknowledged	Verified: `burn()` requires approval from token holder (contract itself), so not directly exploitable.
M-11: Swapper max approval	Downgraded to L-04	Risk limited to allowlisted exchanges during atomic swap. Post-swap approval reset works correctly.

Feb 2026 Finding	Apr 2026 Status	Notes
M-12: Config immediate effect	Downgraded to L-05	Admin-trust issue. Timelock recommended but not a direct vulnerability.

New Findings (not in February audit)

ID	Severity	Title
M-04	MEDIUM	Stale KSU epoch price inflates loyalty levels
M-05	MEDIUM	Loss repayment has no upper bound
M-08	MEDIUM	Withdrawal pro-rata truncation permanently delays small positions
L-01	LOW	Max deposit check only on pending, not total position
L-02	LOW	Deposit round-up dust accumulation
L-06	LOW	Pending deposit tracking vs total position

Notable Methodology Difference

February audit H-01/H-02 (FTD batch array corruption): The previous audit flagged swap-and-pop during reverse iteration as HIGH severity. This audit's State Inconsistency analysis (Phase 5: Parallel Path Comparison) specifically investigated this pattern and determined it is **correct**: reverse iteration processes the last element first, so when an earlier element is removed and the last element swaps in, it has already been processed. The `nextIndexToProcess` is re-initialized at each clearing start via `_initializeLendingPoolFixedTermDepositsProcessing`. We classify the original H-01/H-02 as **false positives**. However, we recommend the Kasu team validate this conclusion with a targeted test case.

K A S U

Redefining Real-World Yield — RWA Private Credit

kasu.finance · Documentation · Twitter/X · Discord · Telegram · Medium

hello@kasu.finance

End of Report — Generated by Claude (Anthropic) — April 9, 2026
Commit: b18e98716c3720e821699f081520484648d6d729 · Branch: release-candidate
Methodology: Feynman + State Inconsistency + Nemesis Fusion